

MODÜL 03

Kayıp Fonksiyonları ve Optimizasyon

Loss Functions & Optimization Algorithms

MSE · MAE · Huber · BCE · CCE · SGD · Momentum · RMSProp · Adam · AdaGrad

Seviye	Orta — Matematiksel Derinlik
Kapsam	6 Kayıp Fonksiyonu · 5 Optimizer · 5 LR Scheduler
Framework	NumPy · TensorFlow/Keras · PyTorch
Önceki Modül	Modül 02 — Aktivasyon Fonksiyonları
Sonraki Modül	Modül 04 — Backpropagation

Modül Konsept Haritası

Kayıp Fonk.		Optimizasyon		LR Scheduler
MSE/MAE/BCE		SGD→Adam		Cosine/Warmup
Regresyon		Problem Seçimi		Sınıflandırma
MSE/MAE/Huber		Görev bazlı		BCE/CCE
NumPy Impl.		TF/Keras		PyTorch
Sıfırdan		compile/fit		optim.Adam

1. Öğrenme Hedefleri

- ✓ MSE, MAE, Huber, BCE, CCE ve KL Divergence kayıp fonksiyonlarını formüle etmek
- ✓ Hangi kayıp fonksiyonunun hangi problem tipine uygun olduğunu seçmek
- ✓ MSE ile BCE arasındaki farkı sınıflandırma bağlamında matematiksel kanıtlamak
- ✓ SGD, Momentum, RMSProp, Adam ve AdaGrad algoritmalarını sıfırdan uygulamak
- ✓ Adam'ın Momentum + RMSProp kombinasyonu olduğunu türevle göstermek
- ✓ 5 farklı öğrenme hızı programını karşılaştırmalı analiz etmek
- ✓ TensorFlow/Keras ve PyTorch'ta kayıp fonksiyonu ve optimizier kullanımını kodlamak

2. Kayıp Fonksiyonları

Kayıp fonksiyonu (loss function), modelin tahminlerinin gerçek değerlerden ne kadar uzakta olduğunu ölçen bir skaldır. Eğitim boyunca bu skalar minimize edilir. Farklı problem tipleri farklı kayıp fonksiyonları gerektirir.

Kayıp Fonksiyonu	Formül	Problem Tipi	Aykırı Değer
MSE	$(1/n) \cdot \sum (y - \hat{y})^2$	Regresyon	Hassas
MAE	$(1/n) \cdot \sum y - \hat{y} $	Robust regresyon	Dayanıklı
Huber	$\frac{1}{2}e^2$ ($ e \leq \delta$) veya $\delta(e - \frac{1}{2}\delta)$ ($ e > \delta$)	Aykırı değerli regr.	Orta
BCE	$-(1/n) \cdot \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	İkili sınıflandırma	—
CCE	$-(1/n) \cdot \sum_i \sum_k y_{ik} \cdot \log(\hat{y}_{ik})$	Çok sınıflı	—
KL Divergence	$\sum P(x) \cdot \log(P(x)/Q(x))$	VAE, dağılım farkı	—

2.1 MSE — Mean Squared Error

$$L = (1/n) \cdot \sum_i (y_i - \hat{y}_i)^2$$

Büyük hatalara kare alarak orantısız ceza verir. Türevi: $\partial L / \partial \hat{y} = (2/n) \cdot (\hat{y} - y)$. Pürüzsüz ve türevlenebilir — optimizasyon kolaylaşır. Dezavantaj: Aykırı değerler (outlier) kare alındığı için kayba büyük katkı yapar.

2.2 BCE — Binary Cross-Entropy

$$L = -(1/n) \cdot \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Neden BCE, MSE'den daha iyi? Örnek: $y=1$, $\hat{y}=0.01$ (çok yanlış tahmin). MSE ceza: $(1-0.01)^2 = 0.98$. BCE ceza: $-\log(0.01) = 4.6$. BCE çok daha büyük ceza → gradyan daha büyük → daha hızlı öğrenir. Sigmoid + BCE kombinasyonu gradyanı basitleştirir: $\partial L / \partial z = (\hat{y} - y)/n$.

2.3 Huber Loss — En İyi İki Birden

Küçük hatalar ($|e| \leq \delta$): $L = \frac{1}{2} \cdot e^2$ [MSE gibi — smooth türev]

Büyük hatalar ($|e| > \delta$): $L = \delta \cdot (|e| - \frac{1}{2}\delta)$ [MAE gibi — lineer büyüme]

δ parametresi 'küçük' ve 'büyük' hatayı ayırır. $\delta=1$ tipik başlangıç. $\delta \rightarrow \infty$ ise Huber $\rightarrow$ MSE davranışı; $\delta \rightarrow 0$ ise Huber $\rightarrow$ MAE davranışı.

3. Optimizasyon Algoritmaları

Optimizasyon algoritmaları, kayıp fonksiyonunu minimize etmek için ağırlıkları nasıl güncelleyeceğimizi belirler. Gradient descent'in temel formülü: $W \leftarrow W - \eta \cdot \nabla W$. Fark, η 'nin ve güncellemenin nasıl hesaplandığındadır.

SGD — Stochastic Gradient Descent

$$W \leftarrow W - \eta \cdot \nabla W$$

- En basit optimizör — herhangi bir DL projesinin başlangıç noktası
- Tüm parametrelere aynı öğrenme hızı uygulanır
- Gürültülü güncellemeler bazen yerel minimumlardan çıkmaya yardımcı olur
- Sorun: Dik vadilerde (ill-conditioned) çok yavaş ilerler

SGD + Momentum

$$v \leftarrow \beta \cdot v - \eta \cdot \nabla W \quad W \leftarrow W + v$$

- $\beta=0.9$ tipik değer — geçmiş gradyanların %90'ı korunur
- Top gibi ivme kazanır — düz bölgelerde daha hızlı ilerler
- Yerel minimumlardan daha kolay çıkar
- β ve η birlikte ayarlanmalı

RMSProp

$$s \leftarrow \rho \cdot s + (1-\rho) \cdot (\nabla W)^2 \quad W \leftarrow W - (\eta / \sqrt{s + \epsilon}) \cdot \nabla W$$

- Her parametreye farklı öğrenme hızı uygular
- Sık güncellenen parametreler $\rightarrow$ lr küçük; nadir güncellenenler $\rightarrow$ lr büyük
- Özellikle RNN eğitiminde popüler (Hinton tarafından önerildi)
- $\rho=0.9$, $\epsilon=1e-8$ tipik değerler

Adam — Adaptive Moment Estimation

$$m \leftarrow \beta_1 m + (1-\beta_1) \nabla W \quad v \leftarrow \beta_2 v + (1-\beta_2) (\nabla W)^2 \quad W \leftarrow W - \eta \cdot m / (\sqrt{v} + \epsilon)$$

- Momentum + RMSProp kombinasyonu — en yaygın kullanılan optimizör
- Bias düzeltmesi: ilk iterasyonlarda sıfıra yaklaşmayı önler
- Varsayılan: $\eta=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$
- Çoğu problemde iyi başlangıç noktasıdır

AdaGrad

$$G \leftarrow G + (\nabla W)^2 \quad W \leftarrow W - (\eta / \sqrt{G + \epsilon}) \cdot \nabla W$$

- G sürekli birikir — lr monoton azalır
- Sparse (seyrek) gradyanlar için iyi — NLP embedding güncellemeleri

- Sorun: Derin ağılarda G çok büyür $\rightarrow lr \rightarrow 0 \rightarrow$ öğrenme durur
- RMSProp bu sorunu çözmek için G yerine hareketli ortalama kullanır

3.6 Optimizer Karşılaştırması

Optimizer	Adaptif lr?	Momentum	Hız	Ne Zaman Kullan?
SGD	Hayır	Hayır	Yavaş	Basit problem, tam kontrol
SGD+Momentum	Hayır	Evet	Orta	CV fine-tuning için
RMSProp	Evet	Hayır	Hızlı	RNN, LSTM eğitimi
Adam	Evet	Evet	Hızlı	Genel — varsayılan tercih
AdamW	Evet	Evet	Hızlı	Transformer, BERT fine-tuning
AdaGrad	Evet	Hayır	Orta	NLP sparse embedding

4. Öğrenme Hızı Programları (LR Schedulers)

Sabit öğrenme hızı her zaman optimal değildir. Eğitim başında büyük adımlar, sonunda küçük adımlar genellikle daha iyi yakınsama sağlar. LR Scheduler bu geçişi otomatik yönetir.

Scheduler	Formül	Kullanım Durumu
Sabit	$\eta(t) = \eta_0$	Basit başlangıç denemeleri
Adım Azalma	$\eta(t) = \eta_0 \cdot \text{drop}^{[t/n]}$	Önceden bilinen epoch sayısı
Üstel Azalma	$\eta(t) = \eta_0 \cdot e^{(-k \cdot t)}$	Sürekli, pürüzsüz azalma
Cosine Annealing	$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})[1 + \cos(\pi t/T)]$	Büyük ölçekli model eğitimi
Warmup + Cosine	Önce lineer artış, sonra cosine	Transformer, büyük batch eğitimi

Warmup neden gerekli? Transformer eğitiminde başlangıçta ağırlıklar rastgele — büyük lr anında büyük ve gürültülü güncellemelere yol açar. Warmup ile lr sıfırdan yavaşça artar, ağırlıklar 'ısındıkça' lr de artar. BERT ve GPT eğitiminde standart yaklaşımdır.

5. Framework Kullanımı

5.1 TensorFlow / Keras

Görev	Kayıp Fonksiyonu Kodu
Regresyon	<code>model.compile(loss='mse')</code>
İkili sınıf.	<code>model.compile(loss='binary_crossentropy')</code>
Çok sınıf (one-hot)	<code>model.compile(loss='categorical_crossentropy')</code>
Çok sınıf (integer)	<code>model.compile(loss='sparse_categorical_crossentropy')</code>
Huber	<code>model.compile(loss=tf.keras.losses.Huber(delta=1.0))</code>

Optimizer	Keras Kodu
SGD	<code>tf.keras.optimizers.SGD(lr=0.01, momentum=0.9)</code>
RMSProp	<code>tf.keras.optimizers.RMSprop(lr=0.001, rho=0.9)</code>
Adam	<code>tf.keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999)</code>
AdamW	<code>tf.keras.optimizers.AdamW(lr=0.001, weight_decay=0.01)</code>
Cosine LR	<code>tf.keras.optimizers.schedules.CosineDecay(0.1, 1000)</code>

5.2 PyTorch

Görev	Kayıp Fonksiyonu Kodu
Regresyon	<code>nn.MSELoss()</code> veya <code>nn.L1Loss()</code> (MAE)
Huber	<code>nn.HuberLoss(delta=1.0)</code>
İkili sınıf.	<code>nn.BCEWithLogitsLoss()</code> [sigmoid dahil — önerilen]
Çok sınıf	<code>nn.CrossEntropyLoss()</code> [softmax + log dahil]
KL Divergence	<code>nn.KLDivLoss(reduction='batchmean')</code>

Optimizer	PyTorch Kodu
SGD	<code>torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)</code>
Adam	<code>torch.optim.Adam(model.parameters(), lr=0.001, betas=(0.9,0.999))</code>
AdamW	<code>torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)</code>
Cosine LR	<code>torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=100)</code>

6. Yaygın Hatalar ve Tuzaklar

Hata 1: Regresyon için BCE kullanmak	BCE çıktının (0,1) aralığında olmasını varsayar. Regresyon çıktısı bu aralıkta değil. MSE, MAE veya Huber kullanın.
Hata 2: Sınıflandırma için MSE kullanmak	MSE, sigmoid'in doyma bölgelerinde çok küçük gradyan üretir — öğrenme yavaşlar. BCE kullanın.
Hata 3: PyTorch'ta BCELoss + Sigmoid VEYA BCEWithLogitsLoss	İkisi aynı sonucu verir ama BCEWithLogitsLoss sayısal olarak daha kararlıdır. Logit üzerinde çalışır.
Hata 4: Adam için lr=0.1 seçmek	Adam'ın varsayılan lr=0.001 genellikle iyi çalışır. lr=0.1 çok büyük olabilir — kayıp salınım yapar.
Hata 5: Scheduler olmadan çok uzun eğitmek	Sabit lr ile uzun eğitim yerel minimumda takılabilir. Cosine annealing veya adım azalma ekleyin.
Hata 6: Her epoch'ta scheduler.step() çağırmaı unutmak (PyTorch)	PyTorch'ta scheduler.step() manuel olarak çağırılmalıdır. Çağrılmazsa lr hiç değişmez.

7. Özet Tablo

Kavram	Formül / Kural	Ne Zaman
MSE	$(1/n) \cdot \sum (y - \hat{y})^2$	Regresyon — standart
MAE	$(1/n) \cdot \sum y - \hat{y} $	Aykırı değerli regresyon
Huber	$\frac{1}{2}e^2$ veya $\delta(e - \frac{1}{2}\delta)$	Hem smooth hem robust
BCE	$-(1/n) \cdot \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	İkili sınıflandırma
CCE	$-(1/n) \cdot \sum y_k \cdot \log(\hat{y}_k)$	Çok sınıflı
SGD	$W \leftarrow W - \eta \nabla W$	Basit başlangıç
Momentum	$v \leftarrow \beta v - \eta \nabla W$; $W \leftarrow W + v$	CV fine-tuning
Adam	$W \leftarrow W - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$	Varsayılan tercih
AdamW	Adam + weight decay	Transformer fine-tuning
Cosine Annealing	$\eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$	Büyük model eğitimi
Warmup + Cosine	Lineer artış + cosine	BERT, GPT eğitimi

8. Kaynaklar

- [1] Kingma, D.P. & Ba, J. (2014). Adam: A Method for Stochastic Optimization. ICLR 2015. arXiv:1412.6980.
- [2] Duchi, J. et al. (2011). Adaptive Subgradient Methods for Online Learning (AdaGrad). JMLR.
- [3] Hinton, G. (2012). RMSProp: Unpublished lecture notes, Coursera Neural Networks course.
- [4] Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR 2019.
- [5] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning, MIT Press. Bölüm 8.
- [6] Ruder, S. (2016). An Overview of Gradient Descent Optimization Algorithms. arXiv:1609.04747.
- [7] PyTorch torch.optim Dokümantasyonu. <https://pytorch.org/docs/stable/optim.html>

Sonraki Adım: Modül 04 — Backpropagation | Zincir Kuralı · Hesaplama Grafi · Sıfırdan Autograd · Vanishing/Exploding